

Embedded Control Systems

Bus structure



Introduction

- A computer bus is a **communication system used to transfer data** between different components of a computer system.

- It connects:

□ CPU

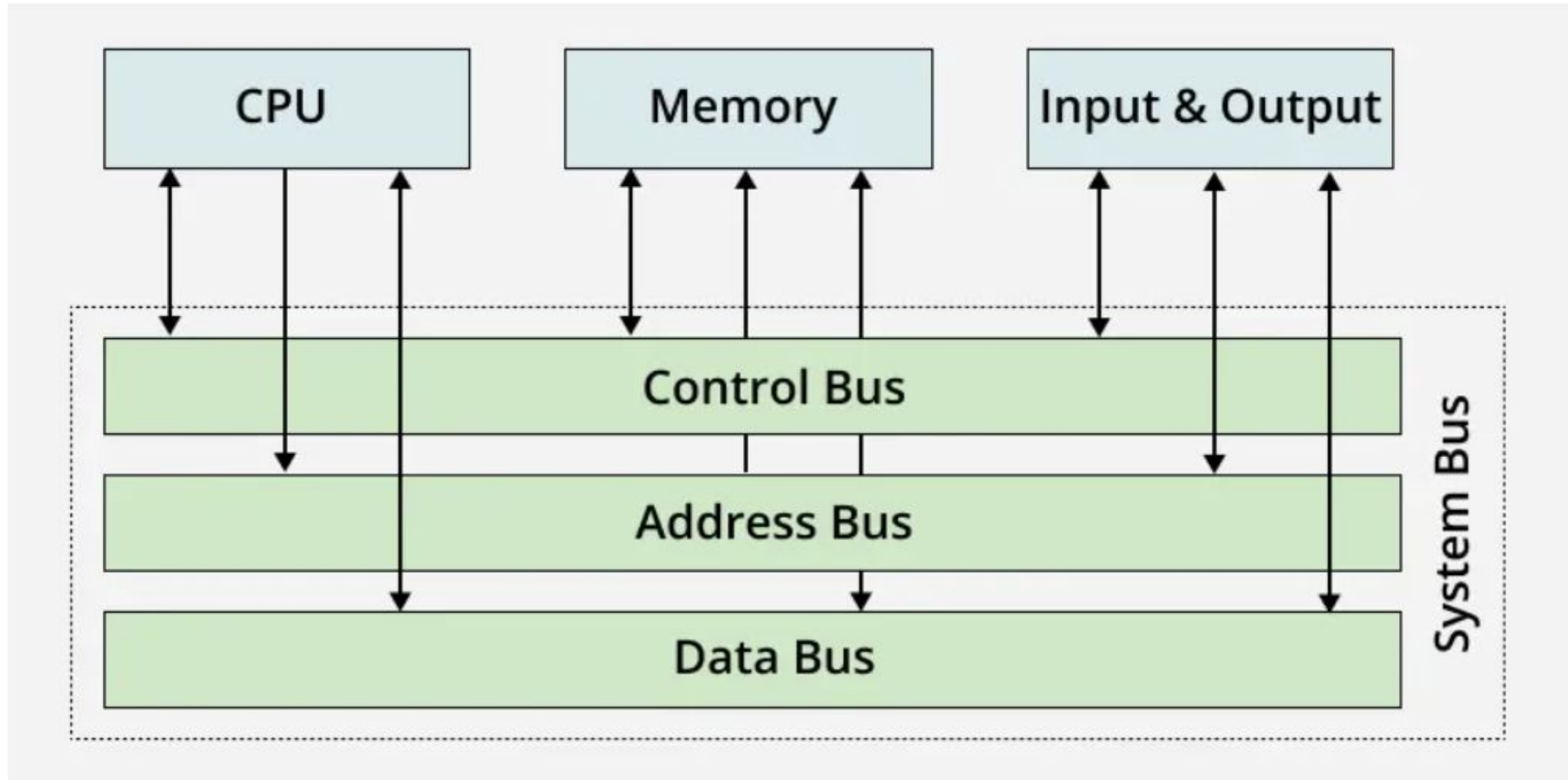
□ Main Memory

□ Input/Output (I/O) devices

- Instead of every component having separate connections with every other component, **the bus provides a shared pathway for communication.**
- It simplifies **data transfer and improves efficiency.**
- It consists of physical connections like **wires, circuits, or cables.**

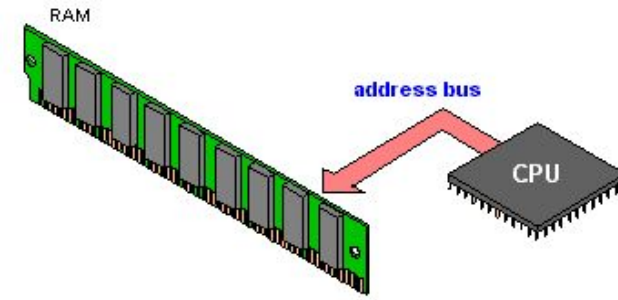


Types of Buses

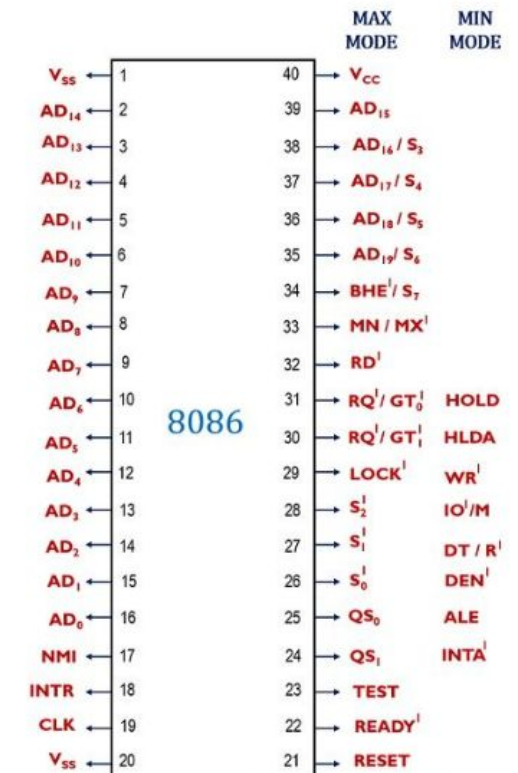


Address Bus

- The **Address Bus** is a collection of wires used to **identify a particular location in main memory**. In simple words, it carries the **address of the memory location** that the CPU wants to access.
- The address **bus transports memory addresses which the processor wants to access in order to read or write data..**
- The address bus is **unidirectional. E.g. CPU → Memory / I/O devices**
- The size of address bus determines how many unique memory locations can be addressed.
- Example:
- A system with 4-bit address bus can address $2^4 = 16$ Bytes of memory.
- A system with 16-bit address bus can address $2^{16} = 64$ KB of memory
- A system with 20-bit address bus can address $2^{20} = 1$ MB of memory. **(in 8086 processor 20 address line= 1MB of memory)**
- **Note- In Shown fig Pin AD0 to AD19 are address bus lines**



The figure below represents the pin diagram of 8086 microprocessor.

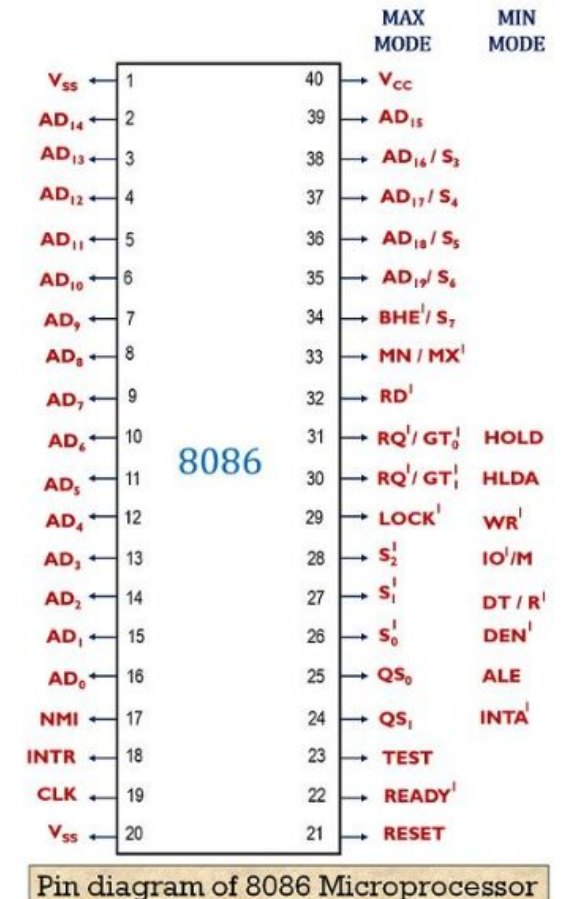


Pin diagram of 8086 Microprocessor

Data Bus

- The **Data Bus carries the actual data and instructions** between: CPU, Memory & I/O Devices
- The data lines **carry binary data between the CPU, memory, and I/O devices** and are **bidirectional**.
- The size (width) of bus determines how much data can be transmitted at one time.
- Example:
- 16-bit bus can transmit 16 bits of data at a time.
- In 8086 Processor it is 16-bit processor, the data bus consists of **16 lines from AD0 to AD15 (Multiplexed Buses)**
- **AD0 to AD15, at T1 acts as address bus & at T2 acts as data bus**

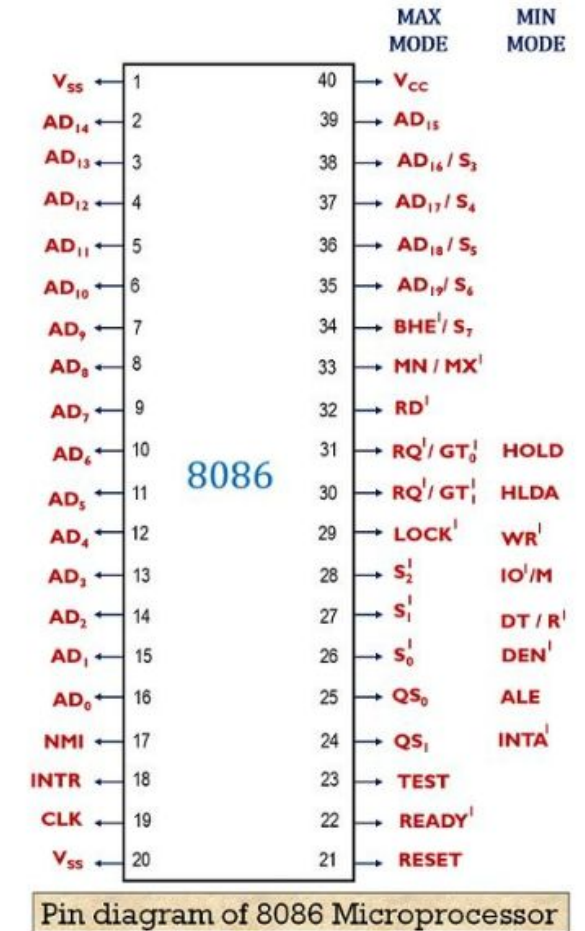
The figure below represents the pin diagram of 8086 microprocessor.



Control Bus

- The Control Bus carries **control and timing signals** between CPU, Memory and I/O Devices
- The main objective of control bus tells **what operation(read or write) to perform and when**
- Control Bus Signal also **manage bus arbitration, allowing multiple devices to communicate efficiently** without conflicts.
- The **Control bus is bidirectional**; the data can flow in either direction from CPU to memory (or I/O device) or from memory to the CPU.
- E.g. 8086 Microprocessor all other pins are control bus pin except Vss & Vcc
- Memory Read Process-
 1. Address place on address bus
 2. M/IO = 1 (Memory operation)
 3. $\overline{RD} = 0$ (Read signal active)
 4. Memory sends data on data bus

The figure below represents the pin diagram of 8086 microprocessor.

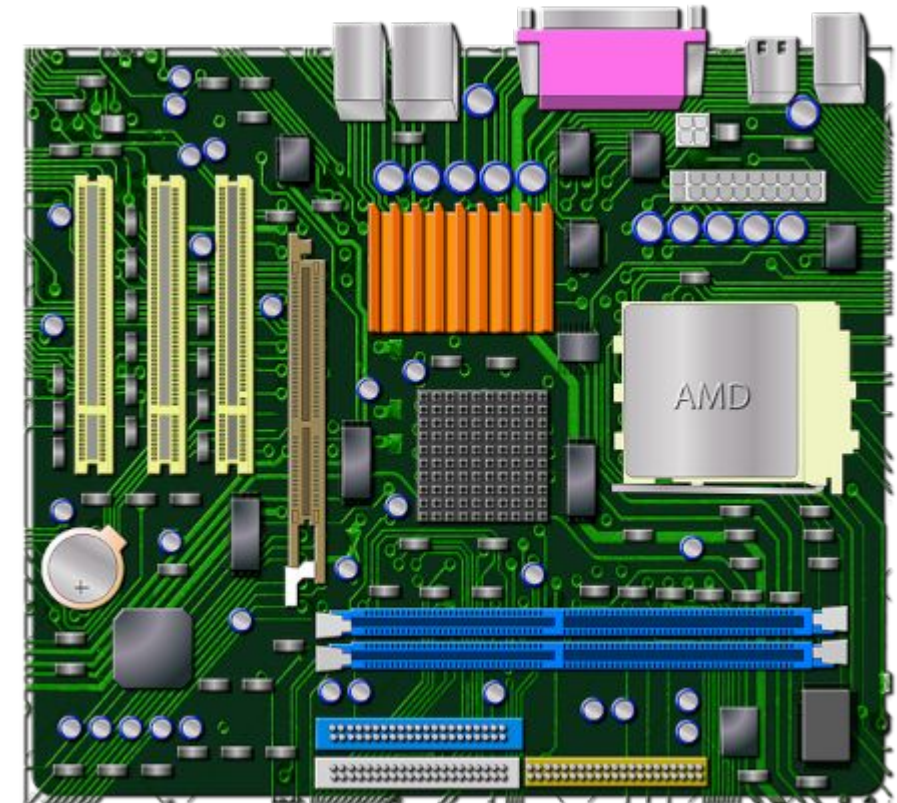


Comparison of Address Bus, Data Bus, and Control Bus

Bus Types	Function	Direction
Address Bus	Carries memory address (location)	Unidirectional
Data Bus	Carries Actual Data	Bidirectional
Control Bus	Carries Control & Timing Signals	Bidirectional

Bus Architecture

- Bus architecture defines **how buses are organized in a computer system.**
- It determines how CPU, memory, and I/O devices communicate.
- It affects; System performance, Scalability, Cost, Complexity
- **Types of Bus Architecture-**
 1. Single Bus Architecture
 2. Hierarchical Bus Architecture



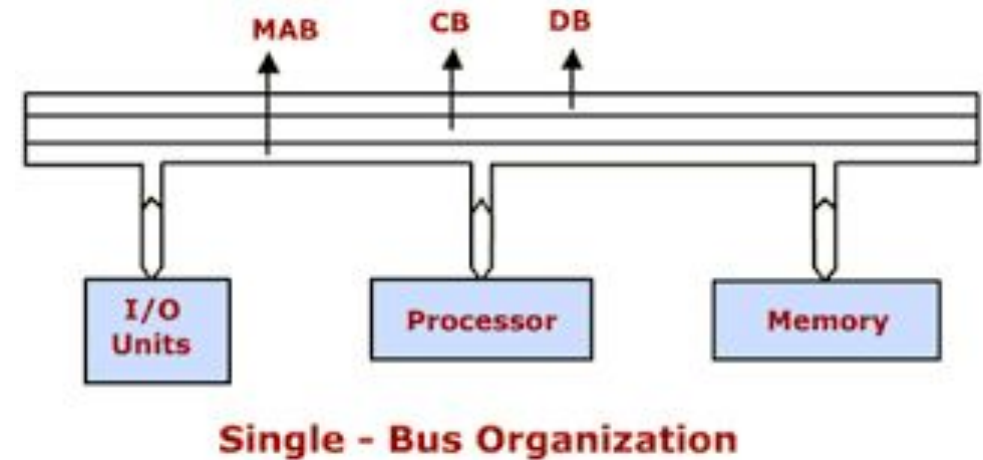
1. Single Bus Architecture

- All components **share one common bus**- CPU, Memory, and I/O devices use the same communication path.
- Only **one device can use the bus at a time**.
- Working Principle-

1. CPU places address on bus.
2. Control signals are sent.
3. Data is transferred.

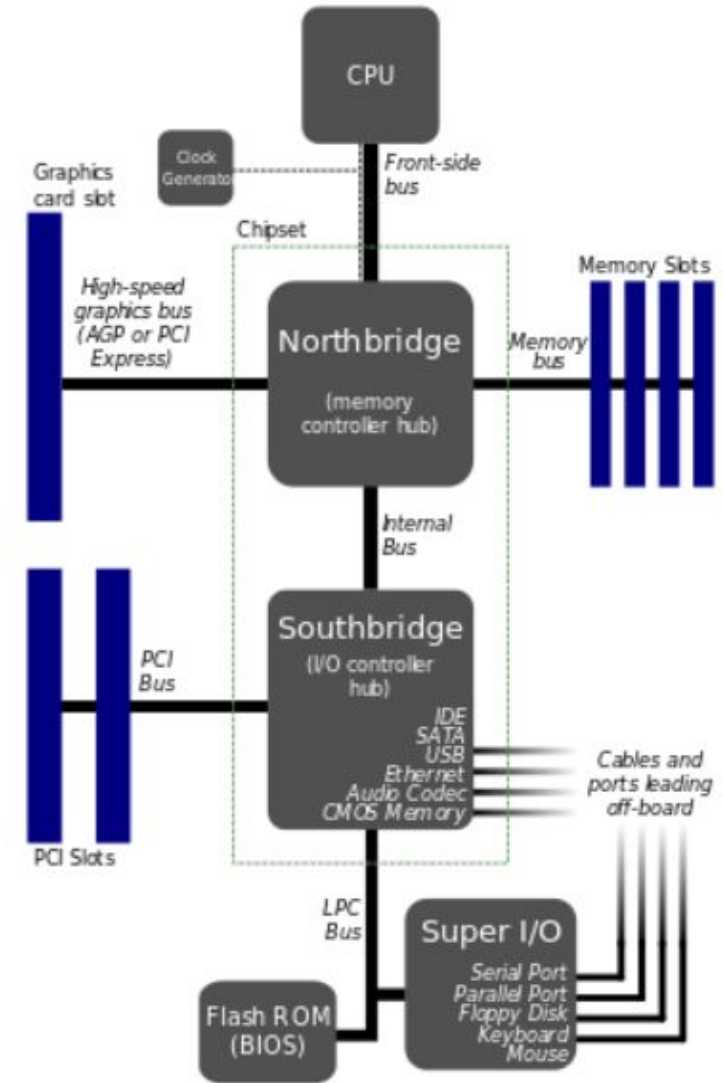
4. Other devices must wait until bus becomes free

- Pros- Low cost, simple design, easy to implement
- Cons- Bus bottleneck, Limited performance, Poor scalability
- Examples- Intel 8085 & 8086



2. Hierarchical Bus Architecture

- It **uses multiple buses arranged in levels** which separates high-speed and low-speed communication.
- **Reduces traffic on main bus.**
- Working Principle-
 1. CPU communicates with memory via high-speed bus.
 2. I/O devices use a separate bus.
 3. Bridges connect different buses.
 4. Multiple transfers can occur efficiently.
- Pros-Higher performance, Reduced bottleneck, Better scalability, Supports multiple devices
- Cons-More complex design, Higher cost, Requires bus controllers/bridges
- Examples- CPU bus, USB bus, SATA Bus etc



Bus Master, Slaves, Arbitration

1. Bus Master

- In Bus System, **Devices that control the Bus** is called as Bus Master
- **Only one master can control the bus at a time.**
- Bus Master Initiates read/write operations, Places address on address bus, Generates control signals & Controls data transfer.
- E.g. CPU, DMA Controller & Bus Controller

2. Bus Slaves

- In Bus System, **Devices that responds to bus master** is called as Bus slaves
- Bus Slaves respond to master request, Sends or receives data, But Cannot initiate communication
- E.g. RAM, ROM, I/O Devices & Peripheral chips

3. Bus Arbitration

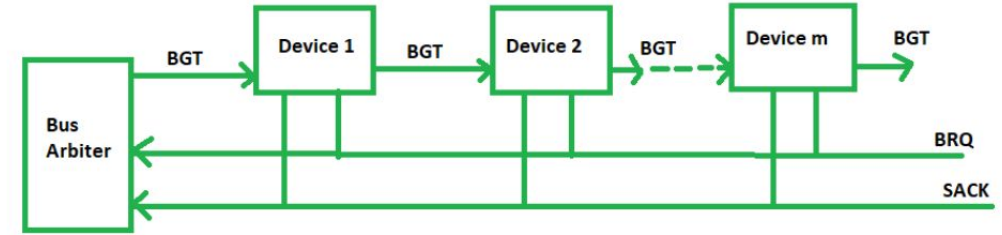
- Bus arbitration is the **process of Selecting one master among multiple requesters** to control the bus.
- If multiple masters try to use the bus simultaneously can cause Data corruption & System instability.
- It ensures fair access, no conflict & organized communication
- Types of Bus Arbitration-
 - a) Centralized Arbitration- **A single bus arbiter performs the required arbitration**
 - b) Distributed Arbitration- **All devices participating in the selection of the next bus master**

Methods of Centralized BUS Arbitration

There are three bus arbitration methods:

1. Daisy Chaining method

- It is **a method where all the bus masters use the same line** for making bus requests.
- It is a simple hardware priority **method used to decide which device gets control of a shared system bus.**
- The bus grant signal serially propagates through each master until it encounters the first one that is requesting access to the bus
- This master blocks the propagation of the bus grant signal; therefore any other requesting module will not receive the grant signal and hence cannot access the bus.
- Pros- Simplicity, Scalability, Low cost & Fast decision
- Cons- Fixed priority, Propagation delay, Failure of one device can break chain



Daisy chained bus arbitration

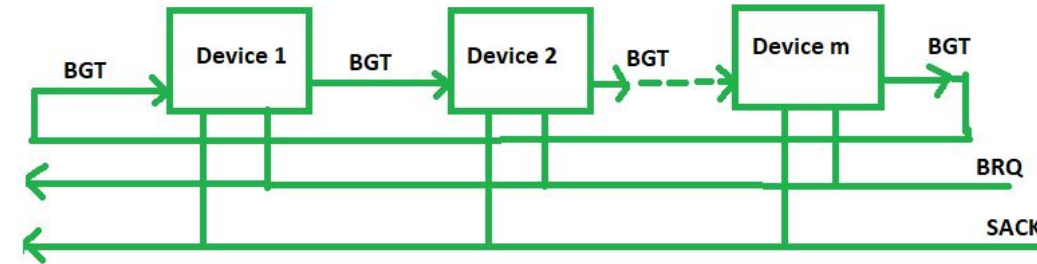
How Daisy Chaining Works-

1. **BRQ** – Bus Request from devices
 - Sent by devices to the **Bus Arbiter**
 - Meaning: I want to use the bus
2. **BGT** – Bus Grant send by Arbiter
 - Passed from device to device (daisy chain)
 - Meaning: You are allowed to use the bus
3. **SACK** – Acknowledge
 - Sent back to arbiter
 - Confirms that some device has taken control

Methods of Centralized BUS Arbitration

2. Polling or Rotating Priority method

- **The bus arbiter gives bus access to devices one by one in a circular order**
- In this, the controller is used to generate the address for the master(unique priority), the number of address lines required depends on the number of masters connected in the system.
- The controller generates a sequence of master addresses. When the requesting master recognizes its address, it activates the busy line and begins to use the bus.
- **Pros-** Method does not favor any particular device, Simple method, If one device fails then the entire system will not stop working
- **Cons-** Adding bus masters is difficult as increases the number of address lines of the circuit



Rotating priority bus arbitration

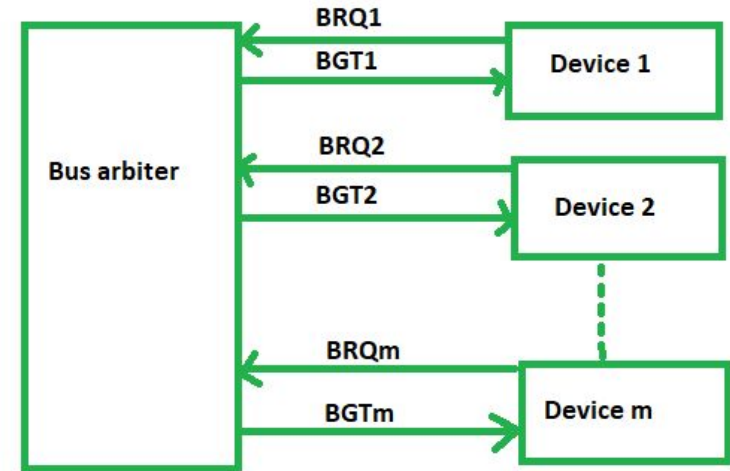
How Polling Method Works-

1. **BRQ** – Bus Request from devices
 - All devices send request on common BRQ line
2. **BGT** – Bus Grant send by Arbiter
 - Grant signal moves in circular order
 - The arbiter keeps track of **who was served last**
3. **SACK** – Acknowledge
 - Device that captures bus sends acknowledgment

Methods of Centralized BUS Arbitration

3. Fixed priority or Independent Request method

- In this, **each devices has a separate pair of bus request and bus grant lines** and each pair has a priority assigned to it.
- The built-in priority decoder within the controller selects the highest priority request and asserts the corresponding bus grant signal.
- Pros-This method generates a fast response.
- Cons-Hardware cost is high as a large no. of control lines is required.



Fixed priority bus arbitration method

How Fixed priority method Works-

1. **BRQ** – Bus Request from devices
 - Each device independently sends request:
2. **BGT** – Bus Grant send by Arbiter
 - The arbiter uses **priority encoder logic**.
 - Grant Given to Highest Priority Device
3. **SACK** – Acknowledge
 - Device that captures bus sends acknowledgment

THANK YOU